Laboratory Series No. 03

Dr. Rochdi Boudjehem

2025-11-01

Lab 2 — OpenMP Worksharing: Reduction, Scheduling, and False Sharing

This lab deepens your OpenMP skills: you will compare reduction strategies, explore loop scheduling policies and chunk sizes under load imbalance, and demonstrate false sharing and its mitigation.

References: Chapter 1 (performance metrics), Chapter 2 (paradigms), and OpenMP basics from lecture.

---

1) Learning Objectives

- Implement and validate OpenMP reductions; compare correctness and performance vs. naive approaches.
- Evaluate scheduling policies (static/dynamic/guided) and chunk sizes under imbalanced workloads.
- Demonstrate false sharing, measure its impact, and eliminate it via padding or privatization.

---

2) Environment & Setup (recap)

- Windows (cmd + MinGW-w64): ensure `...\mingw64\bin` on PATH; `gcc --version`.
- WSL/Linux: `sudo apt install build-essential` (OpenMP is included in GCC).

OpenMP compile/run examples (cmd.exe):

```
set CC=gcc
%CC% -O3 -march=native -fopenmp openmp_lab2.c -o openmp_lab2.exe
set OMP_NUM_THREADS=8
openmp_lab2.exe --mode reduce --N 100000000
```

---

3) Task Description

You will use a single program with multiple modes:

- Mode `reduce`: sum a large array via three variants and compare:
  1) naive global update with `#pragma omp parallel for` + `sum += ...` (data race: incorrect) — only as a demonstration; don't use for timing.
  2) protected update with `#pragma omp parallel for` + `#pragma omp atomic` (correct but slower)
  3) proper OpenMP `reduction(+:sum)` (correct and usually fastest)
- Mode `schedule`: emulate load imbalance using a loop with variable work per iteration; compare scheduling policies:
  - `--schedule static|dynamic|guided` and chunk size via `--chunk k`
  - Measure runtime for each policy and chunk size. Identify best configuration.

- Mode `false`: demonstrate false sharing by updating per-thread counters in a shared array:
  - Without padding: `counters[tid]++` in a tight loop
  - With padding: structure padded to cache line
  - Compare throughput/time.

---

4) Starter Code (C/OpenMP)

Save as `openmp_lab2.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include <omp.h>
#ifdef _WIN32
#include <windows.h>
static double now_s(){ LARGE_INTEGER f,c; QueryPerformanceFrequency(&f); QueryPerf
ormanceCounter(&c); return (double)c.QuadPart/(double)f.QuadPart; }
#else
#include <time.h>
static double now_s(){ struct timespec ts; clock_gettime(CLOCK_MONOTONIC,&ts); ret
urn ts.tv_sec + ts.tv_nsec*1e-9; }
#endif

#ifndef CACHELINE
#define CACHELINE 64
#endif

typedef struct { double v; char pad[CACHELINE - sizeof(double)]; } padded_double;

static void fill_array(double *a, size_t N){
    for(size_t i=0;i<N;i++) a[i] = sin((double)i) * 0.5 + 1.0;
}

static void busy_iters(uint64_t k){ volatile double x=0; for(uint64_t i=0;i<k;i++)
{ x = x * 1.0000001 + 1.0; } (void)x; }

static void set_schedule(const char* pol, int chunk){
    omp_sched_t s = omp_sched_static;
    if(pol){
        if(strcmp(pol,"static")==0) s = omp_sched_static;
        else if(strcmp(pol,"dynamic")==0) s = omp_sched_dynamic;
        else if(strcmp(pol,"guided")==0) s = omp_sched_guided;
        else if(strcmp(pol,"auto")==0) s = omp_sched_auto;
    }
    if(chunk<=0) chunk = 1;
    omp_set_schedule(s, chunk);
}

int main(int argc, char** argv){
    const char* mode = "reduce"; // reduce | schedule | false
    size_t N = 100000000ULL;     // 1e8 by default where applicable
    int threads = 0;             // 0 => use env
    const char* sched = "static";
```

```
    int chunk = 1;
    uint64_t work_unit = 1000;   // variable work in schedule mode
    int repeat = 3;

    for(int i=1;i<argc;i++){
        if(!strcmp(argv[i],"--mode") && i+1<argc) mode = argv[++i];
        else if(!strcmp(argv[i],"--N") && i+1<argc) N = (size_t) strtoull(argv[++i
],NULL,10);
        else if(!strcmp(argv[i],"--threads") && i+1<argc) threads = atoi(argv[++i]
);
        else if(!strcmp(argv[i],"--schedule") && i+1<argc) sched = argv[++i];
        else if(!strcmp(argv[i],"--chunk") && i+1<argc) chunk = atoi(argv[++i]);
        else if(!strcmp(argv[i],"--work") && i+1<argc) work_unit = (uint64_t) strt
oull(argv[++i],NULL,10);
        else if(!strcmp(argv[i],"--repeat") && i+1<argc) repeat = atoi(argv[++i]);
        else if(!strcmp(argv[i],"--help")){
            printf("Usage: %s --mode [reduce|schedule|false] [--N n] [--threads t]
[--schedule static|dynamic|guided] [--chunk k] [--work w] [--repeat r]\n", argv[0]
);
            return 0;
        }
    }
    if(threads>0) omp_set_num_threads(threads);

    if(strcmp(mode,"reduce")==0){
        double *a = (double*) aligned_alloc(64, N*sizeof(double));
        if(!a){ fprintf(stderr,"alloc failed\n"); return 1; }
        fill_array(a,N);
        // ground truth
        double ref = 0.0; for(size_t i=0;i<N;i++) ref += a[i];

        // variant: atomic reduction
        double best_atomic=1e100, best_reduction=1e100; double sum_atomic=0, sum_r
ed=0;
        for(int r=0;r<repeat;r++){
            double s=0; double t0=now_s();
            #pragma omp parallel for
            for(long long i=0;i<(long long)N;i++){
                double v = a[i];
                #pragma omp atomic
                s += v;
            }
            double t1=now_s(); if((t1-t0)<best_atomic){ best_atomic=t1-t0; sum_ato
mic=s; }
        }
        for(int r=0;r<repeat;r++){
            double s=0; double t0=now_s();
            #pragma omp parallel for reduction(+:s)
            for(long long i=0;i<(long long)N;i++) s += a[i];
            double t1=now_s(); if((t1-t0)<best_reduction){ best_reduction=t1-t0; s
um_red=s; }
        }
        printf("mode=reduce N=%zu threads=%d ref=%.4f atomic_sum=%.4f t_atomic=%.6
f s reduction_sum=%.4f t_reduction=%.6f s\n",
               N, omp_get_max_threads(), ref, sum_atomic, best_atomic, sum_red, be
st_reduction);
```

```
        free(a);
    }
    else if(strcmp(mode,"schedule")==0){
        if(chunk<=0) chunk=1; set_schedule(sched, chunk);
        // Emulate imbalance: iterations have cost proportional to (i%100)
        long long M = (long long)N;
        double best=1e100; for(int r=0;r<repeat;r++){
            double t0=now_s();
            #pragma omp parallel for schedule(runtime)
            for(long long i=0;i<M;i++){
                uint64_t k = (uint64_t)((i % 100) + 1) * work_unit; // variable wo
rk
                busy_iters(k);
            }
            double t1=now_s(); if((t1-t0)<best) best=t1-t0;
        }
        printf("mode=schedule N=%zu threads=%d schedule=%s chunk=%d work_unit=%llu
time_best=%.6f s\n",
               N, omp_get_max_threads(), sched, chunk, (unsigned long long)work_un
it, best);
    }
    else if(strcmp(mode,"false")==0){
        int T = omp_get_max_threads();
        padded_double *c_pad = (padded_double*) aligned_alloc(64, T*sizeof(padded_
double));
        double *c_nopad = (double*) aligned_alloc(64, T*sizeof(double));
        if(!c_pad||!c_nopad){ fprintf(stderr,"alloc failed\n"); return 1; }
        for(int i=0;i<T;i++){ c_pad[i].v=0.0; c_nopad[i]=0.0; }
        uint64_t iters = (uint64_t)N; // reuse N as iteration count per thread

        // no padding
        double best1=1e100; for(int r=0;r<repeat;r++){
            for(int i=0;i<T;i++) c_nopad[i]=0.0;
            double t0=now_s();
            #pragma omp parallel
            {
                int tid = omp_get_thread_num();
                for(uint64_t k=0;k<iters;k++) c_nopad[tid] += 1.0; // likely false
sharing
            }
            double t1=now_s(); if((t1-t0)<best1) best1=t1-t0;
        }
        // padded
        double best2=1e100; for(int r=0;r<repeat;r++){
            for(int i=0;i<T;i++) c_pad[i].v=0.0;
            double t0=now_s();
            #pragma omp parallel
            {
                int tid = omp_get_thread_num();
                for(uint64_t k=0;k<iters;k++) c_pad[tid].v += 1.0; // reduced fals
e sharing
            }
            double t1=now_s(); if((t1-t0)<best2) best2=t1-t0;
        }
        // combine results to avoid dead code elimination
        double s1=0,s2=0; for(int i=0;i<T;i++){ s1+=c_nopad[i]; s2+=c_pad[i].v; }
```

```
        printf("mode=false threads=%d iter/thread=%llu sum_nopad=%.0f t_nopad=%.6f
s sum_padded=%.0f t_padded=%.6f s speedup=%.2f x\n",
               T, (unsigned long long)iters, s1, best1, s2, best2, best1/best2);
        free(c_pad); free(c_nopad);
    }
    else{
        fprintf(stderr, "Unknown mode: %s\n", mode);
        return 2;
    }
    return 0;
}
```

Build (cmd.exe):

```
set CC=gcc
%CC% -O3 -march=native -fopenmp openmp_lab2.c -o openmp_lab2.exe
```

Run examples (cmd.exe):

```
set OMP_NUM_THREADS=8
openmp_lab2.exe --mode reduce --N 100000000 --repeat 3
openmp_lab2.exe --mode schedule --N 1000000 --schedule dynamic --chunk 8 --work 50
0 --repeat 3
openmp_lab2.exe --mode false --N 2000000 --repeat 3
```

Tips:

- For `schedule`, tune `--N` and `--work` so T=1 runtime is ~1 s.

- For `false`, keep `N` (iterations per thread) large enough to observe differences.

---

5) Data Collection Templates

1. Reduction table (example headers):

threads	N	t_atomic (s)	t_reduction (s)	speedup (atomic/reduction)	sums_equal?

2. Scheduling table:

threads	schedule	chunk	work_unit	time_best (s)

3. False sharing table:

threads	iters/thread	t_nopad (s)	t_padded (s)	speedup

---

6) Deliverables (due 1 week after lab)

Submit a folder `lab2_openmp_<YourName>` containing:

- `openmp_lab2.c` - `build.cmd` with your build
- `README.md` with:
    - Platform (OS, CPU, RAM), compiler
    - Methods: parameters used (N, schedule, chunk, work_unit)

- Tables for reduction, scheduling, and false sharing - 4–8 bullet observations

---

7) Grading Rubric (10 pts)

- Readiness (2): Builds and runs; environment noted
- Correctness (4): Reduction correctness validated; scheduling mode applied; false sharing mitigation demonstrated
- Performance (2): Coherent trends; chunk/policy impact discussed
- Report (2): Clear tables and observations

---

8) Troubleshooting

- If `aligned_alloc` is unavailable, use `_aligned_malloc`/`_aligned_free` on Windows, or `malloc` without alignment.
- To enforce runtime schedule, compile once and use `omp_set_schedule` with `schedule(runtime)` in the pragma as provided.
- If results are noisy, increase problem sizes or repeat count and report best/avg.
- To pin threads, try setting `set OMP_PROC_BIND=TRUE` on systems where binding helps.

Happy threading — you'll see how policy and memory behavior shape OpenMP performance.